



中国科学院计算技术研究所
INSTITUTE OF COMPUTING TECHNOLOGY, CHINESE ACADEMY OF SCIENCES

NVIDIA CUDA DEEP NEURAL NETWORK LIBRARY

cuDNN: Efficient Primitives for Deep Learning

Zexin Jian

Institute of Computing Technology, Chinese Academy of Sciences

jianzexin25z@ict.ac.cn

Background	2	Convolution	19
Deep Learning	3	Direct Convolution	20
GPU Architecture	5	img2col	21
Why cuDNN?	6	Winograd	22
cuDNN	8	Winograd	23
Related Work	9	FFT	24
Deep Learning Framework	10	Forward	25
Evolution of GPU Acceleration	11	Backward	26
cuDNN	12	Mixing Accuracy & TensorCore	27
cuDNN	13	GPU Memory	28
Key Abstraction: Handle	14	Performance	29
Key Abstraction: Descriptor	15	Performance	30
Algorithm Selection Mechanism	17	Performance	31
Core Functions and Optimizations	18	Performance	32

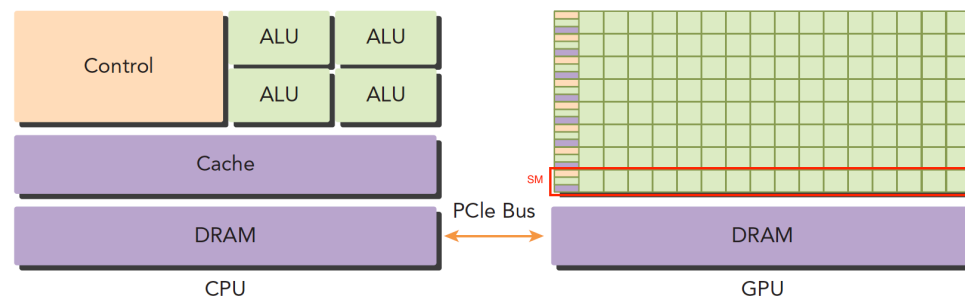
Background



- 模型复杂度爆炸式增长
 - 从 AlexNet 的 6000 万参数到 GPT-3 的 1750 亿参数，深度学习模型规模呈指数级增长。以 ResNet-152 为例，其参数量达 6000 万，单次前向传播需要超过 100 亿次浮点运算
- 训练时间成本高昂
 - 在 CPU 上训练一个 ResNet-50 模型需要数周甚至数月时间。以 ImageNet 数据集为例，使用传统 CPU 集群训练可能需要超过 30 天，严重影响研究迭代速度
- 内存带宽瓶颈
 - 深度学习涉及大量数据搬运，传统 CPU 的内存带宽成为性能瓶颈。卷积操作需要频繁访问内存，导致计算单元经常处于等待状态，利用率低下



- CPU v.s. GPU
 - CPU：设计用于串行处理，拥有少量强大核心（通常 4-64 核），擅长复杂逻辑控制和分支预测，但并行计算能力有限
 - GPU：专为并行计算设计，拥有数千个简单核心（NVIDIA A100 拥有 6912 个 CUDA 核心），适合执行大量相同操作的计算任务
- CUDA
 - > 深度学习中的卷积、矩阵乘法等操作具有天然的并行性，完美契合 GPU 的架构特点。一个卷积层计算可以分解为数千个独立的乘加运算，在 GPU 上可同时执行



Why cuDNN?

- CUDA 编程困难
 - Grid、Block、Warp、Thread、Shared Memory...
 - Simplify ? TileLang、Triton、cuTile...
- 性能优化需要专业知识
 - 需要深入理解 GPU 架构特性，如 warp 调度、内存合并访问、bank conflict 避免
- 算法选择的复杂性
 - 卷积操作有多种实现算法（直接卷积、im2col+GEMM、Winograd 等），每种算法在不同输入尺寸和硬件上性能差异巨大。手动选择最优算法需要大量实验和经验

Why cuDNN?

```
1 import tilelang.language as T
2 # Algorithm Specification
3 M, N, K = 1024, 1024, 1024
4 block_M, block_N, block_K = 128, 128, 32
5 num_stages = 1
6 threads = 128
7
8 def Matmul(A: T.Tensor, B: T.Tensor, C: T.Tensor):
9     with T.Kernel(
10         N // block_N, M // block_M, threads=threads
11     ) as (bx, by):
12         # Buffer Allocation
13         A_shared = T.alloc_shared(block_M, block_K)
14         B_shared = T.alloc_shared(block_K, block_N)
15         C_local = T.alloc_fragment(block_M, block_N)
16
17         # Initialize C_local
18         T.clear(C_local)
19
20         # Main Loop with Pipeline Annotation
21         for k in T.Pipelined(K // block_K, num_stages):
22             T.copy(A[by * block_M, k * block_K], A_shared)
23             T.copy(B[k * block_K, bx * block_N], B_shared)
24             T.gemm(A_shared, B_shared, C_local)
25
26         # Copy the result to the output buffer
27         T.copy(C_local, C[by * block_M, bx * block_N])
28
29 program = Matmul(A, B, C)
30 # Compile
31 kernel = tilelang.compile(program, target="cuda")
32
```

(a) An example TileLang Program

```
@T.prim_func
def Matmul(A: T.Tensor, B: T.Tensor, C: T.Tensor):
    # Buffer Allocation
    A_shared = T.decl_buffer((4096,), "float16", "shared")
    B_shared = T.decl_buffer((4096,), "float16", "shared")
    # Lower fragment buffer to threads
    C_local = T.decl_buffer((128), "float16", "local")
    # Thread binding
    bx = T.thread_binding(128, "blockIdx.x")
    by = T.thread_binding(128, "blockIdx.y")
    tid = T.thread_binding(128, "threadIdx.x")

    # ... Initialize C_local
    for i in T.unroll(128):
        C_local[i] = T.float32(0)

    # Main Loop with Expanded Pipeline
    for i in T.unroll(4):
        T.cp_async(A, A_shared, 16)
    # ... Copy B to B_shared
    T.cp_async_commit(0)

    for ko in range(31):
        T.cp_async_wait(0)
        T.gemm_ss(A_shared, B_shared, C_local,
                  128, 128, 32, 2, 2)

        for i in T.unroll(4):
            T.cp_async(A, A_shared, 16)
        # ... Copy B to B_shared
        T.cp_async_commit(0)
    # ... Compute the last stage
    T.copy(C_local, C)
```

(b) Intermediate Tensor IR



```
#include <tl_templates/cuda/gemm.h>
#include <tl_templates/cuda/copy.h>

__global__ void main_kernel(
    const __half* __restrict__ A,
    const __half* __restrict__ B,
    __half* __restrict__ C,
    int m, int n, int k){
    extern __shared__ __align__(1024) uchar buf_dyn_shmem[];
    half* AShared = ...;
    half* BShared = ...;
    float C_local[128];
    // Initialize C_local
    ...
    // Main Loop with Pipeline
    tl::cp_async_gs<16>(AShared, A);
    tl::cp_async_gs<16>(BShared, B);
    tl::cp_async_commit();
    for (int ko = 0; ko < 31; ++ko) {
        tl::cp_async_wait<0>();
        __syncthreads();
        tl::gemm_ss<128, 128, 32, 2, 2>(AShared, BShared, C_local);
        __syncthreads();
        tl::cp_async_gs<16>(AShared, A);
        tl::cp_async_gs<16>(BShared, B);
    }
    tl::cp_async_wait<0>();
    __syncthreads();
    tl::gemm_ss<128, 128, 32, 2, 2>(AShared, BShared, C_local);

    // Copy C_local to C
    ...
}
```

(c) The generated CUDA code

- **预优化算子库**：提供经过高度优化的深度学习基础算子，包括卷积、池化、归一化、激活函数等，开发者只需调用 API 即可获得极致性能
- **自动算法选择**：内置启发式算法，根据输入尺寸、硬件特性自动选择最优实现算法，无需开发者手动调优
- **跨架构兼容**：自动适配不同代际的 NVIDIA GPU，从 Maxwell 到 Hopper 架构
- **框架集成**：被 TensorFlow、PyTorch 等主流框架深度集成

Related Work



- TensorFlow：采用数据流图计算模型，支持静态图优化
 - TensorBoard 可视化
 - TensorFlow Serving 部署
 - TPU 支持
 - PyTorch：采用动态计算图。简洁的 API 设计和 Pythonic 风格
 - 动态图机制
 - TorchScript 部署
 - 强大的生态系统
 - Caffe：专注于卷积神经网络和计算机视觉
- > **分层设计**：所有主流深度学习框架都深度依赖 cuDNN 作为底层加速库。框架提供高层次的神经网络抽象和自动微分能力，而 cuDNN 负责将计算密集型的操作高效地映射到 GPU 硬件上

2012-2014: 早期 CUDA 实现

在深度学习早期，研究人员需要针对不同的硬件架构手动编写 CUDA 内核实现卷积、池化等操作。需要深入理解 GPU 架构且开发效率低。

1

2014: cuDNN 诞生

NVIDIA 推出 cuDNN 1.0，提供标准化的算子库，高效封装了卷积、池化、归一化等关键操作，让研究者专注于算法设计而非底层指令

2

2017: Tensor Core 革命

Volta 架构引入专门用于矩阵运算的 Tensor Cores。cuDNN 7.0 支持混合精度训练。在保持模型精度的同时带来 2-8 倍的吞吐量提升。

3

2020至今: 持续优化

cuDNN 8.x 引入对 Ampere 架构稀疏计算的支持。支持动态形状和图编译器，使 AI 计算变得更加灵活且极度高效。

4

cuDNN





- Handle (句柄)：库的核心上下文对象，管理所有 cuDNN 操作的状态和资源。每个线程需要创建独立的句柄，确保线程安全
 - 封装了特定的 GPU 设备和 CUDA 流，确保所有操作都在正确的上下文中执行

```
typedef struct cudnnContext *cudnnHandle_t;
```

```
// 创建 cuDNN 句柄  
cudnnHandle_t handle;  
cudnnCreate(&handle);  
// 使用句柄进行操作...  
cudnnDestroy(handle);
```

- **一个线程一个句柄**：cuDNN 句柄不是线程安全的。不能在线程 1 和线程 2 中同时使用同一个 handle 调用函数，否则会内部状态错乱崩溃
- **资源占用**：一个 cudnnHandle_t 占用的额外 CPU/GPU 资源极少

- Descriptor (描述符)：使用多种描述符来封装复杂的数据和操作参数
- 张量描述符：统一描述了 3 到 8 维张量的所有属性，如维度、数据类型（FP32/FP16/INT8 等）、步长和数据布局（如 NCHW/NHWC），使得同一个函数可以处理各种形状的张量

```
typedef struct cudnnTensorStruct *cudnnTensorDescriptor_t;
```

```
// 创建 4D 张量描述符
```

```
cudnnTensorDescriptor_t desc;  
cudnnCreateTensorDescriptor(&desc);  
cudnnSetTensor4dDescriptor(  
    desc, CUDNN_TENSOR_NCHW,  
    CUDNN_DATA_FLOAT,  
    batch, channels, height, width);
```


- 其他描述符：类似地还有 `cudaFilterDescriptor_t`（描述卷积核）、`cudaConvolutionDescriptor_t`（描述卷积操作参数）、`cudaPoolingDescriptor_t`（描述池化操作）等，它们共同构成了 cuDNN 高度参数化的接口

- Workspace：某些算法（如 FFT、Winograd）需要额外的临时内存。cuDNN 提供 API 查询所需工作空间大小，由应用分配并传入
- 启发式算法：
 - 张量尺寸和维度、GPU 架构和显存、数据类型、卷积核大小核步长
- 穷举搜索模式：
 - cuDNN 可以运行所有可用算法并实测性能，选择最快的实现

```
cudnnFindConvolutionForwardAlgorithm(  
    handle, xDesc, wDesc, convDesc,  
    yDesc, requestedAlgoCount,  
    &returnedAlgoCount, &perfResults);
```

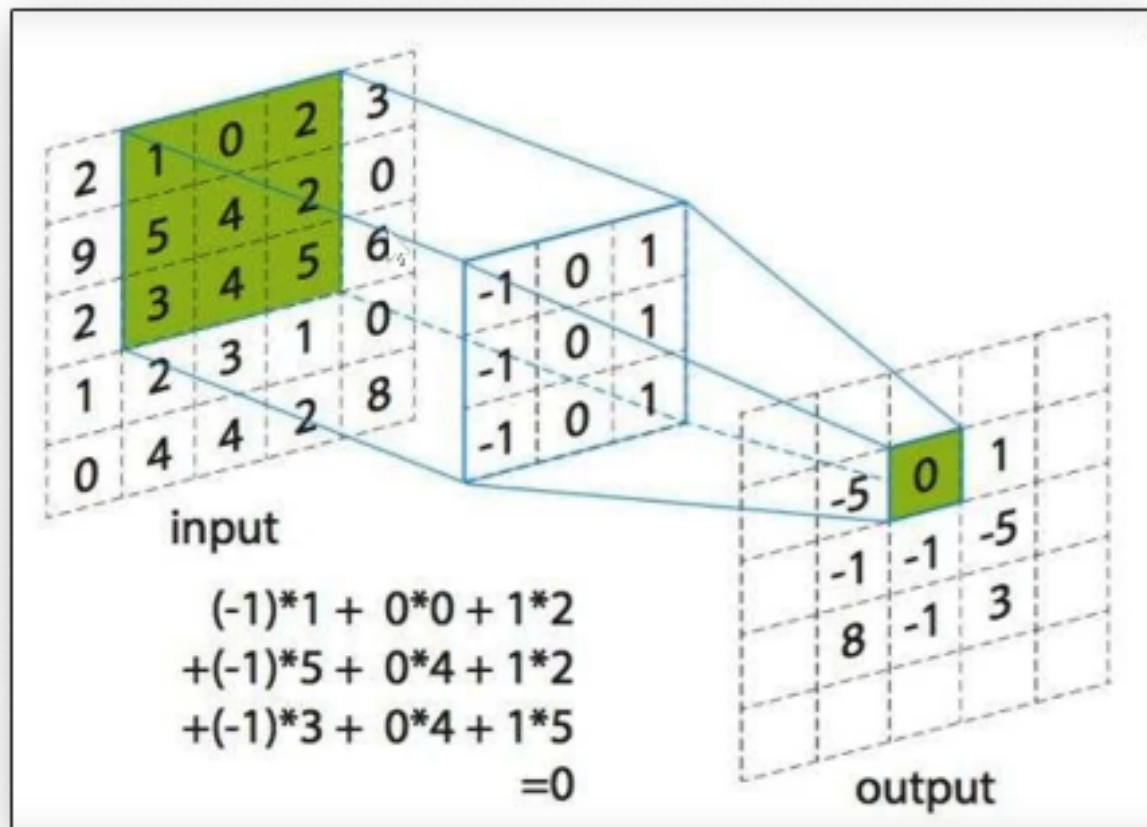
> 对于 cuDNN v8 引入的图 API (Graph API)，它提供了一种更精细的控制方式。你可以获取一个按性能排序的引擎配置列表，然后使用 `cudnnFindPlan` 或自定义函数，从该列表中迭代测试，以找到当前图的最优引擎配置

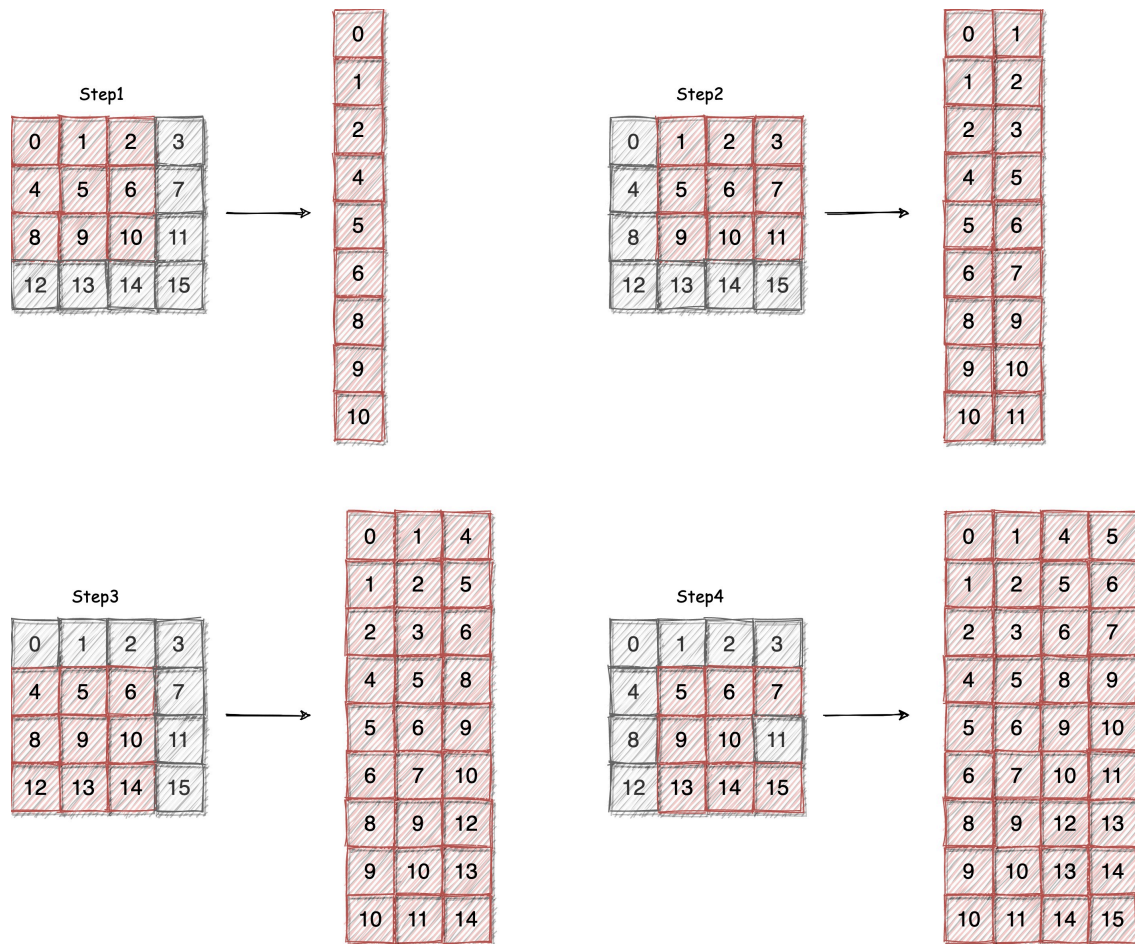


Core Functions and Optimizations

- 直接卷积 (Direct Convolution)：直接在输入特征图上滑动卷积核进行计算。适用于小卷积核 (1×1 , 3×3), 无需额外内存
- img2col+gemm：将卷积展开为矩阵乘法, 利用高度优化的 BLAS 库 (如 cuBLAS)
- Winograd 算法：通过数学变换减少乘法次数, 对小卷积核 (3×3) 特别有效
- FFT 卷积：利用快速傅里叶变换将卷积转换为频域乘法, 对大卷积核特别有效

Direct Convolution





以一维卷积为例,

输入信号

$d = [d_1, d_2, d_3, d_4]^T$

卷积信号

$g = [g_1, g_2, g_3]^T$

$$F(2, 3) = \begin{bmatrix} d_0 & d_1 & d_2 \\ d_1 & d_2 & d_3 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \end{bmatrix}$$

一共需要 6 次乘法和 4 次加法

仔细观察一下，卷积运算中输入信号转换得到的矩阵不是任意矩阵，其有规律的分布着大量的重复元素

$$F(2, 3) = \begin{bmatrix} d_0 & d_1 & d_2 \\ d_1 & d_2 & d_3 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix} = \begin{bmatrix} m_1 + m_2 + m_3 \\ m_2 - m_3 - m_4 \end{bmatrix}$$

其中，在 CNN 的推理阶段，卷积核上的元素是固定的，所以上式中和 g 相关的式子可以提前在模型初始化阶段算好，整个推理阶段只用计算一次，因此可以忽略。所以这里一共需要 4 次乘法 和 8 次加法

$$\begin{aligned} m_1 &= (d_0 - d_2)g_0 & m_2 &= (d_1 + d_2)\frac{g_0 + g_1 + g_2}{2} \\ m_4 &= (d_1 - d_3)g_2 & m_3 &= (d_2 - d_1)\frac{g_0 - g_1 + g_2}{2} \end{aligned}$$

- 当卷积核非常大时，直接卷积计算量会急剧膨胀，FFT 提供了一种从时域 $O(N^2)$ 到频域 $O(N \log N)$ 的转换路径
- 卷积定理：

$$\mathcal{F}(f * g) = \mathcal{F}(f) \cdot \mathcal{F}(g)$$

- 解释：时域里复杂的滑窗、乘加累积，变换到频域后，直接变成了对应元素的相乘
- 计算流程：
 - 变换：分别对信号和卷积核做 FFT ($O(N \log N)$)
 - 点乘：频域复数直接相乘 ($O(N)$)
 - 逆变换：对乘积结果做 IFFT 回到时域 ($O(N \log N)$)

// 卷积前向传播

```
cudaConvolutionForward(  
    handle, &alpha,  
    xDesc, x, wDesc, w,  
    convDesc, algo,  
    workspace, workspaceSize,  
    &beta, yDesc, y);
```

Backward

```
// 数据梯度  
cudnnConvolutionBackwardData(...)  
// 权重梯度  
cudnnConvolutionBackwardFilter(...)  
// 偏置梯度  
cudnnConvolutionBackwardBias(...)
```

- cuDNN 混合精度 API

```
// 设置张量数据类型为 FP16
cudnnSetTensor4dDescriptor(
    desc, format,
    CUDNN_DATA_HALF, ...);
// 启用 Tensor Core
cudnnSetConvolutionMathType(
    convDesc, CUDNN_TENSOR_OP_MATH);
```

- Workspace

```
// 查询 workspace 大小
cudnnGetConvolutionForwardWorkspaceSize(
    handle, xDesc, wDesc, convDesc,
    yDesc, algo, &workspaceSize);
// 分配 workspace 内存
cudaMalloc(&workspace, workspaceSize);
```

- 多 GPU 支持

- cuDNN 本身不直接管理多 GPU，但与 NCCL (NVIDIA Collective Communications Library) 配合实现高效分布式训练

Performance



- on an NVIDIA Tesla K40 of three convolution implementations: cuDNN, Caffe, and cuda-convnet2

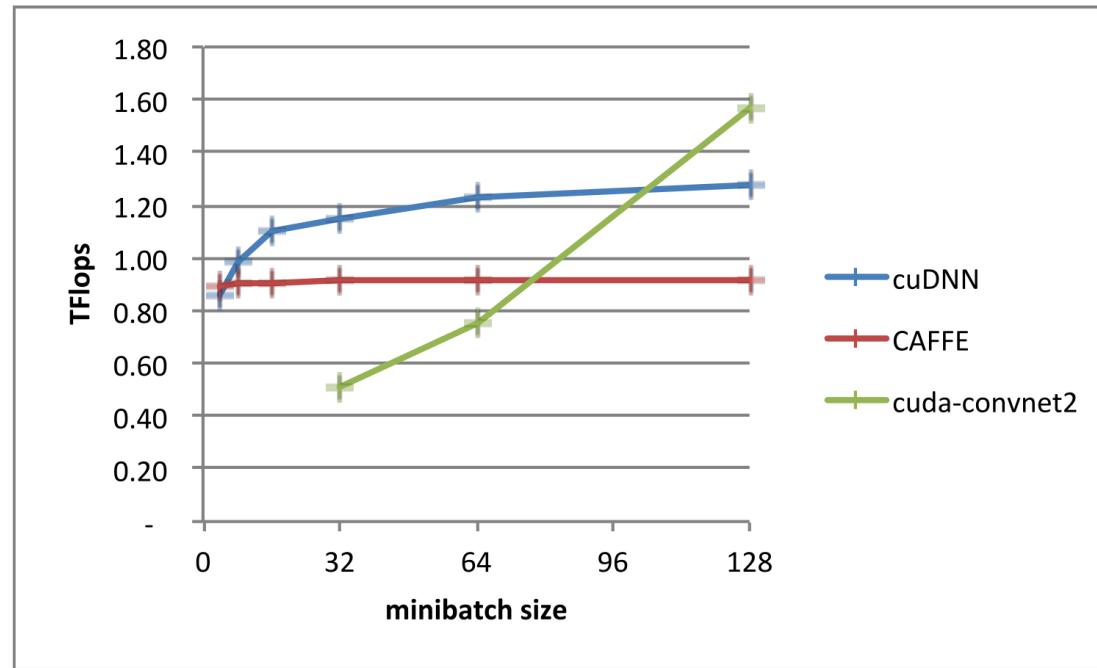


Figure 2: Comparative Performance

- the cuDNN's performance portability across GPU architectures
 - The Tesla K40 is built using the Kepler architecture, and has a peak single-precision throughput of 4.29 TFlops
 - The Geforce GTX 980 is built using the newer Maxwell architecture, and has a peak single-precision throughput of 4.95 TFlops

	K40 (TFlops)	GTX980 (TFlops)	K40 (%)	GTX980 (%)
Layer 1	0.99	1.46	23%	30%
Layer 2	1.52	2.49	35%	51%
Layer 3	1.51	2.42	35%	49%
Layer 4	1.38	2.19	32%	45%
Layer 5	1.01	1.87	24%	38%

Table 3: Performance portability

- the performance improvements gained from integrating cuDNN into Caffe

	Caffe (seconds)	Caffe+cuDNN (seconds)	Speedup
Backward Propagation	156270	120651	1.30
Forward Propagation	109310	75330	1.45
Testing	21600	14529	1.49
Update	2926	2920	1.00
Overall	290106	213431	1.36

Table 4: Caffe Performance

Thank you!

Q & A